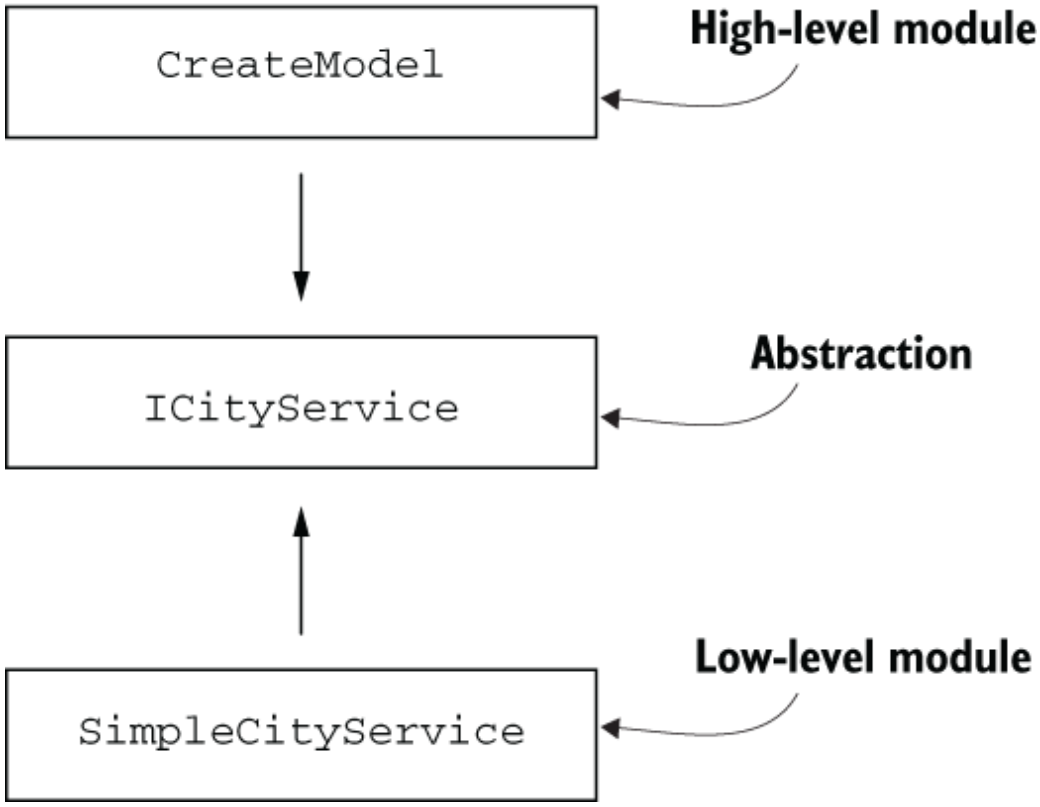


Figure 7.3 The `PageModel` and the `SimpleCityService` depend on an abstraction: the `ICityService` interface.

Now that the dependency chain is *inverted*, you need to design your `ICityService` interface. The second part of the DIP states that the interface should depend on abstractions too, not "details." That is, the interface should not be tied to specific implementations. So your interface should not return implementation-specific types, like a `DbDataReader`, which only works with relational databases. It should depend on more general-purpose types like a `List<T>`. Your `SimpleCityService` class already does that, fortunately. So you will create an interface based on its existing API.

Add a new C# code file to the Services folder, and name it ICityService.cs. Note that if you are using Visual Studio, the Add... New Item dialog includes